

Nowy ekran `/axes`: długość fizyczna, punkt bazowania, limity programowe i przełożenie posuwu dla osi X, Y, Z. Konfiguracja trafia do pliku JSON i jest egzekwowana przy walidacji programów, w ruchu ręcznym i w mostku. Model i uzasadnienie: [../konfiguracja-osi.md](#).

Pliki

- `server/app/axes.py` — nowy: model osi, zakres fizyczny z długości i punktu bazowania, walidacja, zapis/odczyt pliku, obszar roboczy z limitów.
- `server/app/config.py` — `AXES_FILE` (`AXES_CONFIG`); `WORK_AREA` degraduje się do wartości startowych, używanych tylko dopóki nie ma pliku.
- `server/app/main.py` — `GET/PUT /api/axes`, `/axes`, obszar roboczy i ostrzeżenia z konfiguracji osi; `/api/config` zwraca też osie.
- `server/app/machine.py` — konfiguracja osi w warstwie maszyny, kontrola limitu przy JOG, wysyłka `AXCFG` do mostka po połączeniu i po zmianie.
- `server/app/static/axes.html`, `axes.js`, `style.css` — ekran: tabela osi, walidacja na bieżąco, paski zakresów, blokada zapisu w trakcie ruchu.
- `server/app/static/index.html`, `editor.html` — odnośnik do ekranu.
- `bridge/sc4hub_bridge.cpp` — komenda `AXCFG`, przełożenie i limity per oś, odrzucanie ruchu poza limit bez alarmu.
- `server/tests/test_axes.py`, `confptest.py` — testy modelu, pliku i API.
- `start.sh`, `start.bat` — `AXES_CONFIG=../config/axes.json`.

Uwagi

- **Ruch poza limit nie jest awarią** — mostek odrzuca komendę (`Reject`) zamiast wchodzić w `ALARM`, więc dojechanie do granicy zakresu nie wymaga `RESET-u`.
- Przy okazji poprawione: ruch ręczny nie zmienia już `NOT_HOMED` na `READY`. Wcześniej jeden JOG przed bazowaniem wystarczał, żeby `START` przyjął program w nieznanym układzie współrzędnych.
- Zmiana przełożenia wymaga **weryfikacji skoku przejazdem kontrolnym** — ani serwer, ani mostek nie mają jak sprawdzić, czy wpisana wartość odpowiada mechanice.
- Uszkodzony plik konfiguracji przerywa start serwera zamiast podstawiać wartości domyślne.

Źródło: [docs/zmiany/ekran-konfiguracji-osi.md](#) — maszyna do ocinania wlewków płytek optyki